

动态规划

《算法设计与分析》

信息科学与技术学院 曾华琳



05



最大子段和问题



最大子段和问题

问题描述：

给定由n个整数组成的序列 $a_1, a_2, \dots, a_n$ ，求该序列子段和的最大值。

当所有整数均为负值时定义其最大子段和为0。

依此定义，所求的最优值为：

$$\max \left\{ 0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k \right\}$$

例如，当 $(a_1, a_2, a_3, a_4, a_5, a_6) = (-2, 11, -4, 13, -5, -2)$ 时，最大子段和为：

$$\sum_{k=2}^4 a_k = 11 + (-4) + 13 = 20$$



最大子段和问题

一个简单算法:

```
int MaxSum(int n, int a[ ], int &besti, int &bestj) {  
    int i, j, k, sum=0;  
    for (i=0; i<n; i++)  
        for (j=0; j<n; j++) {  
            int thissum=0;  
            for (k=i; k<=j; k++) thissum+=a[k];  
            if (thissum>sum) {  
                sum=thissum;  
                besti=i+1;  
                bestj=j+1;  
            }  
        }  
    return sum;  
} // 时间复杂度为 $O(n^3)$ 
```



最大子段和问题

简单算法的改进:

```
int MaxSum(int n, int a[], int &besti, int &bestj) {  
    int i, j, sum=0;  
    for (i=0; i<n; i++) {  
        int thissum=0;  
        for (j=i; j<n; j++) {  
            thissum+=a[j];  
            if (thissum>sum) {  
                sum=thissum;  
                besti=i+1;  
                bestj=j+1;  
            }  
        }  
    }  
    return sum;  
} // 时间复杂度为 $O(n^2)$ 
```



最大子段和问题

分治算法:

将最大子段和 $a[1:n]$ 分为 $a[1:n/2]$ 和 $a[n/2+1:n]$ 两段,
则可能有下列三种情况:

- 1) $a[1:n]$ 的最大子段和= $a[1:n/2]$ 的最大子段和;
- 2) $a[1:n]$ 的最大子段和= $a[n/2+1:n]$ 的最大子段和;
- 3) $a[1:n]$ 的最大子段和= $\sum_{k=i}^j a_k$, $1 \leq i \leq n/2$, $n/2+1 \leq j \leq n$

此时, 可分别计算 $a[1:n/2]$ 和 $a[n/2+1:n]$ 的最大子段和

$$s1 = \max_{1 \leq i \leq n/2} \sum_{k=i}^{n/2} a[k], \quad s2 = \max_{n/2+1 \leq i \leq n} \sum_{k=i}^n a[k] \quad \rightarrow s1+s2$$



最大子段和问题

分治算法:

```
int MaxSubSum(int a[ ], int left, int right) {  
    int sum=0;  
    if (left==right) sum=a[left]>0 ? a[left] : 0;  
    else {  
        int center=(left+right)/2;  
        int leftsum=MaxSubSum(a,left,center);  
        int rightsum=MaxSubSum(a,center+1,right);  
        int s1=0,lefts=0;  
        for (int i=center; i>=left; i--) {  
            lefts+=a[i];  
            if (lefts>s1) s1=lefts;  
        }  
    }
```



最大子段和问题

分治算法(续):

```
int s2=0,rights=0;
for (i=center+1; i<=right; i++) {
    rights+=a[i];
    if (rights>s2) s2=rights;
}
sum=s1+s2;
if (sum<leftsum) sum=leftsum;
if (sum<rightsum) sum=rightsum;
}
return sum;
}
```

复杂性分析:

$$T(n) = \begin{cases} O(1) & n \leq c \\ 2T\left(\frac{n}{2}\right) + O(n) & n > c \end{cases}$$
$$T(n) = O(n \log n)$$



最大子段和问题

动态规划算法:

在上述分治算法中,

若记 $b_j = \max_{1 \leq i \leq j} \left\{ \sum_{k=i}^j a_k, 1 \leq j \leq n \right\}$, 则所求的最大子段和 为

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k = \max_{1 \leq j \leq n} \max_{1 \leq i \leq j} \sum_{k=i}^j a_k = \max_{1 \leq j \leq n} b_j$$

由 b_j 的定义易知, 当 $b_{j-1} > 0$ 时, $b_j = b_{j-1} + a_j$, 否则, $b_j = a_j$ 。由此可得, 计算 b_j 的动态规划递归式 $b_j = \max\{b_{j-1} + a_j, a_j\}, 1 \leq j \leq n$ 。据此, 可设计出求最大子段和的动态规划算法如下:



最大子段和问题

动态规划算法:

```
int MaxSum(int n, int a[ ]) {  
    int i, sum=0, b=0;  
    for (i=0; i<n; i++) {  
        if (b>0) b+=a[i];  
        else b=a[i];  
        if (b>sum) sum=b;  
    }  
    return sum;  
} // 时间复杂度为O(n)
```

谢谢大家